

Beyond Mesonet4: Benchmarking Modern Deepfake Detection Across Generational Gaps

Ryan Powers

Bellini College of Artificial Intelligence, Cybersecurity, and Computing
University of South Florida
rp194@usf.edu

I. PROJECT GOAL

The goal of this project was to explore how MesoNet4 — one of the earliest deepfake detectors — compare to modern deep learning architectures and techniques when tasked with detecting fakes across both GAN-based and diffusion-based generation methods. Deepfake and manipulated media pose a serious threat to both national and global security. With modern technology being democratized and more accessible than ever, familiarizing ourselves and experimenting with technologies to counter media manipulation to preserve authenticity and integrity of information is becoming increasingly important.

To begin our research into this topic, we implemented Mesonet4 as our baseline architecture. After understanding the capabilities, and limitations, of this legacy model for deepfake detection, we continued our research by experimenting with alternative CNN-based architectures and features, and then used this to compare against with modern day SOTA designs such as ViT-b16. The ultimate goal of this project was to explore the traditional techniques used to detect manipulated media and the improvements made over the years, and use this knowledge to continue our research in the same trajectory as the constant advancements being used for this purpose. Additionally, we sought to find the architectures or features that *generalises across multiple generational forgery methods*. We seek to determine:

- 1) Can a traditional CNN detector (Mesonet4/MesoInception4) remain effective against modern-day deepfake techniques?
- 2) Does the addition of modern features or architectures improve detection and generalization across forgery methods?
- 3) Will the addition of modern features or techniques to legacy CNN-based architectures result in competitive performance against modern architectures?

II. INPUT AND GROUND TRUTH

A. Input Format

All images are resized to 256×256 pixels (3-channel RGB). This resolution is dictated by the MesoNet architecture [1], which was designed for face-crop inputs at this scale. Images whose shortest side is smaller than 256 px are *excluded* rather

than upscaled, as upscaling introduces artificial high-frequency artifacts that can bias CNN feature maps. The only change from our proposal was that for our foundation model, ViT-B16 [2], required the images to be resized to 224px, which was done inline with fine-tuning and evaluation.

B. Ground Truth Labels

Binary labels are used throughout:

- **0 — Real:** unmanipulated face images from real video frames (FF++) or real photographs (IMDB-WIKI via DeepFakeFace, Celebdfv2 Youtube real).
- **1 — Fake:** synthetically generated or manipulated face images from any method (GAN, Diffusion, etc).

III. DATASET DESCRIPTION

The goal was to train, evaluate, and test generalization across three different datasets [listed below]:

A. Dataset #1: GAN-Based Dataset (FF++)

Derived from FaceForensics++ [3], which contains videos manipulated by four GAN-based methods: Deepfakes, Face2Face, FaceSwap, and NeuralTextures. 16 frames were extracted from each real video, while 4 frames were extracted from each fake video from each GAN-forgery method. Extracted frames totaled approximately 16 000 images were split with a 70/15/15 train/test/val.

B. Dataset #2: Diffusion-Based Dataset

This dataset was custom built from several different sources. The intended purpose of this dataset was provide us insights into how each architecture would generalize against this dataset when trained on Dataset #1, as well as generalization against Dataset #1, when the models were trained on Dataset #2. This dataset consists of 51,150 images, perfectly balanced between real and fake. The dataset was split with a 70/15/15 train/test/val. Fake images were generated from techniques including Freedom_T, HPS, MidJourney, SDXL, Freedom_I, DreamBooth, SDXL_Refine, DiffFace, CoDiff, Imagic, cycle_diff, text2img, and inpainting. Custom built from the below sources:

- **DiFF** [4]: A collection of diffusion-generated face images covering multiple sub-categories (HPS, SDXL, DreamBooth, SDXL-Refine, Imagic). Only images with a shortest side ≥ 256 px are retained.

- **DeepFakeFace** [5]: Provides text-to-image fakes (Stable Diffusion 1.5) and inpainting fakes, with real images from IMDB-WIKI celebrities serving as the real class.
- **FaceForensics++** [3]: Real face images drawn from the genuine video frames of this dataset supplement the real class.
- **Celeb-DF-v2** [6]: Additional real celebrity face images sourced from the real-video splits of this benchmark further balance the real class.

C. Held-Out Generalization Set

Celeb-DF-v2 [6] is withheld entirely from training and validation for the official evaluations against models trained on dataset#1 FF++. It serves as an unseen-domain test to measure cross-dataset generalisation.

a) *Implementation Note.*: The final and correct evaluation presented in this work were conducted on models trained on the FF++ dataset. We did include dataset #2 training and evaluation results to show our efforts but these results should not be considered for grading. For dataset#2, models reached ~99% AUC within the first couple of epoch of training and after several iterations of reconstructing the dataset and re-training, we were not getting expected results. I included more information in Section XIII.

IV. DATA PROCESSING

A. Frame Extraction(from sources with mp4)

Frames were extracted using OpenCV's 'VideoCapture'. The quantity of frames extracted from each mp4 file depended primarily on how many images were needed for the particular partition or deepfake technique the video was derived from. For example, dataset#1 was created solely from FaceForensics+ so, in order to have a balanced dataset, we needed to extract less frames from each individual forgery technique, which would add up to the amount we extracted from the real videos. The sampling strategy ensured uniform temporal coverage, without bias — frame indices were computed with `numpy.linspace(0, total_frames-1, n_frames)`.

B. Face Detection and Alignment

MTCNN [7] was used to detect and align faces, with a 20 % margin around each detected face. If MTCNN could not detect a face, the image would be opened with PIL and cropped by `crop_size = min(width, height)`, and then crops equal margins from both sides of the longer dimension. This center-crop fallback was a valid option since all sourced datasets already guarantee facial structures to be centered but we decided to still use the face detection algorithm to ensure best results.

C. Resolution Filtering

Any image whose shortest side is < 256 px was discarded. This was to ensure the images in our dataset were of the best quality and to prevent the model learning artifacts created for smaller images requiring to be upsampled.

D. Balanced Sampling

Each deepfake-generation method had the exact amount of images in each dataset and split. This prevented model bias towards any particular deepfake generation method by ensuring that no single method held more weight during training, validation, or evaluation.

E. Train / Validation / Test Split

Each dataset is split into 70 % train, 15 % validation, and 15 % test, honoring the splits provided by the dataset source (if provided). Furthermore, any image or video from the data sources provided train/test/val split would go into the same split of our custom train/test/val splits.

F. Identity Leakage

FF++ and most diffusion sources included identity metadata by either providing dataset splits or including identification numbers in the video name. For Celeb-dfv2, the Youtube-Real partition does not have any identity metadata but the creator guarantees that the same person is not found in more than one provided video. Considering this, the Youtube-Real videos were split 70/15/15 separately (240/45/45 videos) and then frames were extracted from each split to add into the real partition of Diffusion Dataset #2.

G. Augmentation

Applied during training only:

- ColorJitter: brightness=0.2, contrast=0.2, saturation=0.1
- JPEG Compression: 70 to 95
- GaussianBlur: kernel_size=3, p=0.3
- Per-channel normalisation: Computed stats for the respective training dataset
- For ViT-b16 pretrained: Resize(224) and ImageNet stats

Validation and test sets receive only center-crop and normalization. For the pre-trained ViT, images were all resized to 224x224.

V. METHODOLOGY

A. Baseline Model

We used the original Mesonet4 and MesoInception4 architecture by Afchar *et al.* [1] as our baseline. MesoNet was designed specifically for detecting face manipulations in video, and its compact design made it suitable for rapid experimentation and comparison. Mesoinception4 is a variant of Mesonet, with the first two convolutional blocks being replaced with inception blocks.

Mesonet4 takes a $256 \times 256 \times 3$ input, passes it through four convolutional blocks, with each convolution followed by batch normalization and max pooling. The convolutional blocks are followed by successive fully connected heads with dropout(0.5) and a sigmoid output.

The Mesoinception4 network replaces the first two convolutional blocks of Mesonet4 with two *inception-like* blocks (each block uses parallel 1×1 , 3×3 , and 5×5 convolutions) followed by two plain convolutional layers, fully connected heads with a sigmoid output.

B. CNN Model Architecture

My proposed model augments MesoInception4 with Convolutional Block Attention Modules (CBAM) [8]. CBAM applies two sequential attention sub-modules after a convolutional layer:

- 1) **Channel Attention**: a squeeze-and-excitation style gate that re-weights feature channels by pooling spatial information (both average and max) and passing through a shared MLP.
- 2) **Spatial Attention**: a gate that re-weights spatial locations by pooling channel information and convolving with a 7×7 kernel.

CBAM modules are inserted after each of the two inception blocks and the remaining two convolutional blocks, giving the network the ability to progressively focus on semantically informative regions (e.g. eye region, mouth, blending boundaries). The architecture that had the best performance is summarized in Table I.

TABLE I: MesoInception4-CBAM Architecture Summary

Block	Output Size	CBAM
Inception Block 1	$128 \times 128 \times 11$	✓
Inception Block 2	$64 \times 64 \times 12$	✓
Conv Block 3	$32 \times 32 \times 16$	✓
Conv Block 4	$8 \times 8 \times 16$	✓
Flatten	1024	
FC ($1024 \rightarrow 16 \rightarrow 1$)	1 (logit output)	

My partners implemented the following architectures:

- 1) **Hieu Nguyen**: LVNet and the baseline Mesonet4
- 2) **Tajaddin Gafarov**: MesoXceptionCBAM and MesoXceptionNet

C. Architecture Comparison

Table II compares our best model against partner architectures and pretrained foundation model baselines fine-tuned on the same FF++ training set.

TABLE II: Architecture Comparison — FF++ Test AUC

Model	Params	Pretrained
MesoNet4 (Hieu Nguyen)	~27K	No
MesoInception4 (Ryan Powers)	~27K	No
MesoInception4-CBAM (Ryan Powers)	~28K	No
MesoXceptionCBAM (Tajaddin Gafarov)	~19K	No
MesoXceptionNet (Tajaddin Gafarov)	~18K	No
LvNet (Heiu Nguyen)	~63M	No
Xception [9]	22.9M	Yes
ViT-B/16 [2]	86M	Yes

D. Training Details and Reproducibility

All experiments are implemented in PyTorch. Key hyperparameters for my implementation are summarised in Table III.

Early stopping is applied with patience of 10 epochs, monitoring validation AUC. Model checkpoints are saved whenever validation AUC improves and every 10 epochs. All experiments were run on a single NVIDIA GPU with 16gb VRAM.

TABLE III: Training Hyperparameters

Hyperparameter	Value
Optimiser	AdamW (with default beta1 and beta2)
Learning rate	$5e-3$
Learning rate schedule	ReduceLROnPlateau with linear warmup (optional arg)
Weight decay (L2)	1×10^{-4}
Batch size	64
Epochs	50
Loss function	Binary cross-entropy with logits
Input resolution	256×256

VI. PERFORMANCE IMPROVEMENT TECHNIQUES

Performance improvements were found with the following techniques:

- **CBAM attention**: detailed in Section V-B.
- **JPEG compression augmentation**: to simulate social-media compression and improve generalization
- **L2 regularisation (weight decay)**: reduces over-fitting on the smaller diffusion dataset.
- **Learning rate schedule**: Performance scheduling via `Torch.ReduceLROnPlateau`
- **Dropout**: Dropout(0.3-0.5) depending on the other regularization techniques and parameters.

TABLE IV: Results from finetuning and testing hyperparameter techniques

Model Variant	FF++ AUC	Δ vs. best
MesoInception4 (baseline)	0.7621	−0.048
MesoInception4-CBAM (best)	0.8100	—
No CBAM in Inception block 1	0.7406	−0.069
No geometric augmentation	0.7005	−0.110
No augmentation	0.5772	−0.233
No dropout	0.6749	−0.135
No BatchNorm	0.5004	−0.310
No L2 regularisation	0.6663	−0.144
100 epochs (over-trained)	0.6379	−0.172

BatchNorm and augmentation are load-bearing. Removing BatchNorm causes complete training failure (AUC 0.50, predicted all real). Removing all augmentation drops performance near chance (AUC 0.58). Geometric augmentation gave a −0.11 AUC penalty when removed from specific experiments with attention mechanisms, suggesting that spatial variability during training aids generalization.

VII. PERFORMANCE EVALUATION

A. Metrics

We report the following metrics for all experiments below.

- **Accuracy**: overall classification accuracy.
- **AUC-ROC**: area under the receiver operating characteristic curve — the primary metric, as it is threshold independent.
- **Precision / Recall / F1**: to expose class-specific performance.

B. Training and Evaluation Information

This table includes the results from all successful experiments from each team member. Please note the following:

- Each entry that is trained on FF++ should be considered for evaluation purposes, as the FF++ dataset is verifiably reliable and our concerns regarding Dataset #2 (Diffusion) explained in section X.
- We included the experiments on Dataset#2 (Diffusion) to show our efforts towards training and evaluating on different deepfake techniques to see if our architectures and techniques saw improvements with learning artifacts from the different forgery methods and generalization.

VIII. COMPARISON OF APPROACHES

A. Generalization

The generalization problem is universal, including the fine-tuned foundational models. **Every model fails to generalize on different deepfake methods.** Every model trained on GAN-era deepfakes and tested on diffusion deepfakes has around a 0.45-0.59 AUC, which is effectively guessing. Similar performance was observed for cross-GAN generalization (Celeb-dfv2), with the best results is the pre-trained ViT at 0.64AUC. MesoInception4 trained on Diffusion and tested on FF++ collapses to 0.479 AUC with a recall of 0.009 — it’s predicting nearly everything as real. These results show that the models are learning artifacts that are created from specific deepfake generation methods, and the artifacts are not shared between methods or generations of deepfakes.

B. CBAM Impact Analysis

The impact of CBAM is inconsistent. For my implementation, mesoinception4-cbam, it improved the models performance against regular mesoinception4 on in-domain deepfakes (+ 0.048 AUC and F1) but it decreased performance when the FF++ trained model was evaluated against Celebdf-v2. For MesoXception-net, the separable convolutions does not appear to benefit the model, likely because the attention mechanism may be adding too much noise throughout the model or overfitting. The results show that the benefits of CBAM is architecture and data dependent but generally suggests that the network’s attention to spatially localised artifacts is beneficial.

C. Xception dominates in-domain

Fine-tuned Xception achieves 0.9747 AUC, which shows the advantage of ImageNet pretraining and larger models. However, this comes at roughly $800\times$ the parameter cost ($\sim 28K$ for mesoinception4 vs. 22.9M). The performance of this model in our experiments proves why it has been historically considered the best deepfake detection technique.

D. MesoXceptionNet and MesoXceptionCBAM

These architectures are the weakest on FF++, with MesoXceptionNet at 0.720 AUC and MesoXceptionCBAM at 0.632. These results hint at maybe an architectural compatibility or training issue. The seperable convolutions sacrifice capacity fort efficiency and maybe the early convolutions are not extracting

powerful enough features maps for CBAM to spatially or channelly concentrate on.

E. LvNet

LVnet appears to perform the best on in-domain evaluations (when compared to the CNN-basedx models that were not pretrained), with a 0.896 AUC for FF++. The performance of this model shows the benefits of a two-stream architecture designed for deepfake detection.

F. Mesonet4 Baseline

The baseline architecture ended up being the weakest model against FF++, at 0.606AUC. These results show that the legacy architecture is not robust against data compression and augmentation. This also shows that the addition of inception blocks provide critical enhancements in performance, when comparing to the performance of mesoinception4.

IX. FOUNDATION MODEL COMPARISON

The foundation models include ViT-B16, Resnet18, and Xception. Each model was downloaded via timm as pretrained, and fine-tuned using dataset #1 (FF++).

A. ViT-B16

ViT generalized best out of the pre-trained models, suggesting that the global self-attention captures more facial inconsistencies than the CNN kernels. ViT achieved in-domain 0.9428 AUC and 0.6438 AUC when evaluated against celeb-dfv2. This suggests that even a powerful model with 86M parameters struggles with identifying artifacts across different deepfake generation methods.

B. ResNet-18

Powerful in-domain, which proves that ImageNet pretraining gives a significant advantage over random initialization.

C. Xception

Fine-tuned Xception achieves 0.9747 AUC, which shows the advantage of ImageNet pretraining and larger models. However, this comes at roughly $800\times$ the parameter cost ($\sim 28K$ for mesoinception4 vs. 22.9M). The performance of this model in our experiments proves why it has been historically considered the best deepfake detection technique.

X. ERROR AND FAILURE ANALYSIS

A. Confusion Analysis

On the FF++ test set, MesoInception4-CBAM achieves 0.7183 accuracy (AUC 0.8100) with recall 0.7527 and precision 0.7043. The primary failure mode is an elevated false-negative rate: the model misses fake images more often than it misclassifies real ones (recall < precision gap is consistent across training runs). Per-method, FaceSwap and NeuralTextures are the hardest GAN methods to detect; Deepfakes-style identity-swap artifacts are comparatively easier.

Table VII The below table compares per-class accuracy on the FF++ test set across all four architectures I implemented, trained on FF++. FaceSwap and NeuralTextures are the hardest

TABLE V: In-distribution results: models evaluated on the same dataset they were trained on, sorted by AUC.

Model	Dataset	AUC	Acc.	Prec.	Rec.	F1
LVNet	Diffusion	0.9991	0.9842	0.9770	0.9917	0.9843
MesoInception4-CBAM	Diffusion	0.9935	0.9527	0.9702	0.9341	0.9518
MesoInception4	Diffusion	0.9926	0.9470	0.9494	0.9442	0.9468
Xception (pretrained)	FF++	0.9747	0.9179	0.9457	0.8866	0.9152
ResNet-18	FF++	0.9578	0.8861	0.8643	0.9160	0.8894
ViT-B/16	FF++	0.9428	0.8688	0.8918	0.8393	0.8648
LVNet	FF++	0.8960	0.8217	0.8310	0.8105	0.8206
MesoInception4-CBAM	FF++	0.8100	0.7183	0.7043	0.7527	0.7277
MesoInception4	FF++	0.7621	0.6933	0.7104	0.6527	0.6803
MesoXceptionNet	FF++	0.7195	0.6473	0.6315	0.7071	0.6672
MesoXceptionCBAM	FF++	0.6322	0.5941	0.5813	0.6732	0.6239
MesoNet4	Diffusion	0.6199	0.5171	0.5101	0.8725	0.6438
MesoNet4	FF++	0.6063	0.6020	0.5840	0.6300	0.6061

TABLE VI: Generalization results: models evaluated on a held-out dataset different from their training set, sorted by AUC.

Model	Train → Eval	AUC	Acc.	Prec.	Rec.	F1
ViT-B/16	FF++ → Celeb-DF v2	0.6438	0.4044	0.8369	0.1151	0.2023
ViT-B/16	FF++ → Diffusion	0.5850	0.5422	0.6032	0.2468	0.3503
MesoInception4	FF++ → Celeb-DF v2	0.5677	0.5208	0.7107	0.4552	0.5550
Xception (pretrained)	FF++ → Celeb-DF v2	0.5663	0.4201	0.7361	0.1815	0.2912
Xception (pretrained)	FF++ → Diffusion	0.5017	0.4807	0.4666	0.2695	0.3416
MesoInception4-CBAM	Diffusion → FF++	0.4945	0.4964	0.2500	0.0036	0.0070
MesoInception4-CBAM	FF++ → Celeb-DF v2	0.4813	0.4886	0.6454	0.4902	0.5572
MesoInception4	Diffusion → FF++	0.4792	0.4987	0.4348	0.0089	0.0175
MesoInception4-CBAM	FF++ → Diffusion	0.4731	0.4721	0.4466	0.2330	0.3062
MesoInception4	FF++ → Diffusion	0.4556	0.4970	0.4937	0.2333	0.3168
LVNet	FF++ → Celeb-DF v2	0.570	0.49	–	–	0.485

manipulation methods for every model, while Deepfakes and Face2Face are easier. Xception performed best with an overall 0.918 ACC.

B. Generalization Error

Every model that fails to generalise does so in the same way: it calls almost everything real. Precision stays relatively high — when the model does predict fake, it is usually correct — but recall collapses, meaning the vast majority of fakes are silently misclassified as real. Table VIII shows this breakdown across all four architectures evaluated on Celeb-DF-v2.

The pattern is counter-intuitive: the larger and stronger the model, the *more extreme* the collapse. ViT and Xception are nearly perfect on real images yet almost completely blind to unseen fakes. The compact MesoInception4 models, while weaker in-domain, are at least closer to random on out-of-distribution data. This suggests that higher-capacity models over-fit more tightly to the GAN-specific artefacts present in FF++ training data, leaving them with no signal when those artefacts are absent.

Generalisation to Celeb-DF-v2 breaks down entirely for both MesoInception variants, as visualised in Figure 1. The confusion matrices confirm the asymmetric failure: a large off-diagonal mass of fakes predicted as real, with comparatively few real images mislabelled.

Data scale mismatch. ViT was originally pretrained on ImageNet-21k (>14M images) and relies on task-specific data to fine-tune its attention patterns effectively. The FF++ training set ($\approx 16K$ images) is insufficient to adapt 86M parameters

without overfitting, regardless of the low learning rate. Deepfake detection datasets are orders of magnitude smaller than the data regimes where ViT excels.

Inductive biases matter. Convolutional models like MesoInception4-CBAM have built-in spatial locality and biases well-suited to the task: deepfake artifacts are local, spatially consistent features (blending boundaries, texture discontinuities). ViT’s global self-attention has no such prior and must learn locality from scratch on our limited data.

XI. TEAM COLLABORATION

As a team, we worked together to design the pipeline and the experiments we wanted to perform. Each of us performed different experiments but most of them were related in some way. This helped us learn and understand which architecture or techniques helped detect deepfake artifacts and why.

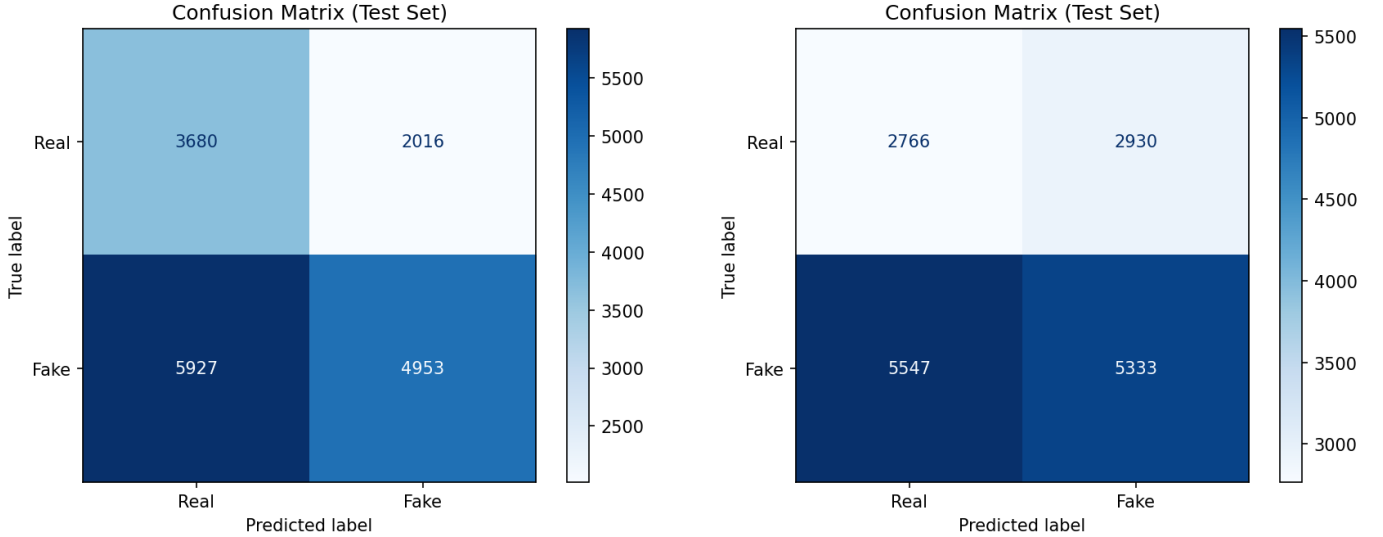
XII. INDIVIDUAL CONTRIBUTIONS

A. Myself

- Designed and constructed Dataset #1 and Dataset #2 and associated manifest files.
- Took responsibility for troubleshooting diffusion dataset #1
- Implemented mesoinception4 and mesoinception4 variants.
- Fine-tuned and evaluated the foundation models: ViT-B16 and Xception.

TABLE VII: Per-Class Test Accuracy on FF++ (all models trained on FF++). Best result per row in **bold**.

Class	MesoInception4	MesoInception4-CBAM	ViT-B/16	Xception
Deepfakes	0.843	0.864	0.886	0.893
Face2Face	0.757	0.779	0.879	0.896
FaceSwap	0.446	0.714	0.882	0.946
NeuralTextures	0.564	0.654	0.711	0.811
Real	0.734	0.684	0.898	0.949
Overall	0.693	0.718	0.869	0.918



(a) MesoInception4 (AUC 0.568, Recall 0.455). Predicts real correctly 64% of the time but misses over half of all fakes.

(b) MesoInception4-CBAM (AUC 0.481, Recall 0.490). CBAM adds no generalisation benefit; performance degrades to near-random.

Fig. 1: Confusion matrices for models trained on FF++ and evaluated on the held-out Celeb-DF-v2 generalisation set. Both models exhibit the same failure mode: they call the majority of unseen fakes *real*, confirming that GAN-trained detectors over-fit to generation-specific artefacts. CBAM does not improve and slightly worsens cross-dataset generalisation (AUC 0.568 $\rightarrow$ 0.481).

TABLE VIII: Real vs. fake accuracy and recall on Celeb-DF-v2 (all models trained on FF++). Higher model capacity correlates with more severe fake-recall collapse.

Model	Real Acc.	Fake Acc.	Recall
ViT-B/16	95.5%	11.5%	0.115
Xception	85.6%	18.2%	0.182
MesoInception4	64.0%	45.5%	0.455
MesoInception4-CBAM	48.6%	49.0%	0.490

B. Hieu

- Implemented Mesonet4 and LVnet
- Implemented and evaluated Resnet18
- Evaluated models for generalization

C. Tajaddin

- Implemented and evaluated MesoxceptionNet and MesoXceptionCBAM
- Finetuned and evaluated Resnet18

XIII. CONCLUSION

We presented a study of deepfake detection generalisation across GAN and diffusion generation families using an attention-enhanced MesoInception4 architecture. Our key findings are:

- CBAM improves in-domain but not generalization. Although the findings were inconsistent across CBAM additions between partners, CBAM generally appears to hurt generalization by decreasing AUC.
- All models fail to generalize and exhibit the same collapse: high real-accuracy, near-zero fake recall on generalization evaluation.
- As model size increases, they magnitude over overfitting increases: ViT with 86M parameters achieves 11.5 percent fake accuracy on celeb-dfv2, while mesoinception4-CBAM with 28k parameters achieves 49 percent. These results show that complex models are not always more effective than smaller models.
- Generalizing on diffusion-generated deepfakes is harder than cross-GAN generalization

- Neural Textures appears to be the hardest method to detect. This is due to the subtle artifacts this method leaves behind, compared to the larger spatial artifacts from the other methods like Deepfakes and Face2face.
- MesoInception4-CBAM achieves better accuracy at $\approx 3,000\times$ fewer parameters and trains in under 2 minutes per epoch versus 70 seconds per epoch for ViT on the same hardware. For constrained deployment scenarios, the compact model is the clear practical choice.

I learned a lot about dataset design and all of the different deepfake techniques. With our diffusion dataset2, Every architecture was achieving .95+ AUC: I knew there was a problem but I did not know exactly what. Originally, I thought it had to do with the original diffusion dataset we were using being perfectly balanced but the real partition only having real images from IMDB-WIKI. I thought a possible solution would be adding real images from multiple sources to potentially prevent bias, so I added real images from the FF++ dataset and CelebDFv2 dataset. The new real partition consisted of .35 Celeb-Youtube, .25 celeb-real, .25 from faceforensics videos, and .15 from IMDB-wiki. After re-training and evaluating, each model still achieved extremely high AUC within the first epochs. While the issue is not fully understood, I do know that the combining techniques introduced a lot of opportunity for bias by combining images and frames extracted from videos, from various sources. I plan on continuing on with combining all of the real and fake images that I have downloaded and creating a detailed manifest so that the training results will show me exactly which deepfake method or source the real or fake image is coming from.

Future work includes continuing to research the modern day, SOTA architectures being used for deepfake forensics and detection. I plan on finishing the data problem we faced to learn more about data engineering, the different deepfake generation methods, and how the models studied in this project and beyond perform. I plan on experimenting with other datasets that I found throughout troubleshooting our dataset issue, that has a great distribution of deepfakes from all generations, across all modalities, to see how the models we trained during this project perform. Ultimately, I hope to discover techniques that enable a model to learn artifacts from all manipulation methods and I want to find the best model size vs speed vs accuracy trade off.

REFERENCES

- [1] D. Afchar, V. Nozick, J. Yamagishi, and I. Echizen, "MesoNet: a compact facial video forgery detection network," in *IEEE International Workshop on Information Forensics and Security (WIFS)*, 2018, pp. 1–7.
- [2] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *International Conference on Learning Representations (ICLR)*, 2021.
- [3] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "FaceForensics++: Learning to detect manipulated facial images," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1–11.
- [4] Y. He *et al.*, "DiFF: Diffusion-based face forgery dataset," in *ArXiv preprint*, 2023.
- [5] Anonymous, "DeepFakeFace: A large-scale dataset for diffusion-generated face detection," *ArXiv preprint*, 2023.
- [6] Y. Li, X. Yang, P. Sun, H. Qi, and S. Lyu, "Celeb-DF: A large-scale challenging dataset for DeepFake forensics," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 3207–3216.
- [7] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, p. 1499–1503, Oct. 2016. [Online]. Available: <http://dx.doi.org/10.1109/LSP.2016.2603342>
- [8] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.
- [9] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251–1258.